



COM410 Programming in Practice

T3. Java Generics



Java Generics

- **Generic methods** allow us to specify operations that can take place on objects of different data types
 - For example, a method that operates on an array might be appropriate on an array of either integers, floating point values or strings – how do we define the parameter?

```
void doSomething (int[] anArray) { ... }  
void doSomething (float[] anArray) { ... }  
void doSomething (char[] anArray) { ... }
```

Java Generics

- The answer is to use the Generic Type **T**, which is introduced in the method signature (before the return type) and then used where required, so...

```
public static <T> void doSomething (T[] anArray) {  
    for (T element : anArray) {  
        System.out.println(element);  
    }  
}
```

- The method can then be called with parameters of any appropriate type, such as...

```
Integer[] intArray = { 1, 2, 3, 4 };  
String[] stringArray = { "one", "two", "three", "four" };  
doSomething(intArray);  
doSomething(stringArray);
```

Java Generics

- A **Generic Class** is specified by providing the generic type after the class name and allows us to define ADTs that can be applied to elements of any type.

```
public class Thing<T> {  
    T aThing;  
  
    public T get () {  
        return this.aThing;  
    }  
  
    public void set (T something) {  
        this.aThing = something  
    }  
}
```

Java Generics

- We provide the type when an instance of the generic class is created

```
public class Thing<T> {  
  
    ...  
  
    public static void main(String[] args) {  
        Thing<String> myThing = new Thing<String>();  
        Thing<Integer> yourThing = new Thing<Integer>()  
  
        myThing.set("Adrian");  
        yourThing.set(10);  
    }  
}
```

Demonstration

- Use generics to provide a single method that will print the contents of either an array of integers or an array of strings

```
public class Printer() {  
  
    public static void main(String[] args) {  
        Integer[] intArray = { 1, 2, 3 };  
        String[] stringArray = { "one", "two", "three" };  
        printArray(intArray);  
        printArray(stringArray)  
    }  
  
    public static void printArray() { }  
}
```

Can you provide
this method?

A Bag of Things

- Our Bag ADT has previously been defined as holding a specific (and defined) type - e.g. a Bag of Strings, a Bag of Buildings, etc.
 - We now want to provide a single Bag definition that can hold a collection of any type of object
 - A generic class

BEFORE – a Bag of Buildings

```
public class Bag {  
    private Building[] bag;  
    ...  
}  
  
Bag b1 = new Bag();
```

AFTER – a Bag of Things

```
public class Bag<T> {  
    private T[] bag;  
    ...  
}  
  
Bag<Building> b1 = new Bag<Building>();  
Bag<String> b2 = new Bag<String>();
```

Generic Constructor

- One of the tasks of our constructor was to define the array that stored the **Bag** contents
- The decision to use a generic data type **T** in the declaration of the bag affects how we will allocate memory for this array within the constructor

```
this.bag = new T[capacity]; // would produce a syntax error
```

- Need to declare an array of type **Object** and cast (convert) it to the desired type

```
T[] tempBag = (T[])new Object[capacity];  
this.bag = tempBag;
```

Note: casting is the conversion of one data type to another, by preceding the variable to be cast by the new type in parenthesis

Demonstration

- Convert the **Bag** class to store a collection of generic objects
- Test the class by defining separate bags of **Integer** and **String** objects and check that you can add, remove and check bag items in each case.